

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №3

Тема: Модульне тестування програмного забезпечення за допомогою xUnit та аналіз покриття коду (Code Coverage)

1. Вихідний код реалізованого модуля з коментарями

Нижче наведено виправлений та структурований код основних класів системи (Documents, ExportFile, ExtractionJob):

```
/// <summary>
    /// Представляє завантажений користувачем документ для подальшої обробки.
    /// Відповідає за зберігання метаданих файлу та їх базову валідацію.
    /// </summary>
    public class Document
    {
        public Guid DocId { get; set; } = Guid.NewGuid();

        public string FileName { get; set; }

        public double FileSizeMB { get; set; }

        public string Format { get; set; }

        /// <summary>
        /// Перевіряє документ на відповідність бізнес-правилам (наявність імені, розмір, формат).
        /// </summary>
        /// <exception cref="ArgumentException">Викидається, якщо ім'я файлу порожнє.</exception>
        /// <exception cref="InvalidOperationException">Викидається, якщо розмір перевищує 50 МБ.</exception>
        /// <exception cref="NotSupportedException">Викидається, якщо формат не підтримується.</exception>
        public void Validate()
        {
            // Перевірка наявності імені файлу
            if (string.IsNullOrEmpty(FileName))
```

```

    {
        throw new ArgumentException("Ім'я файлу не може бути порожнім.");
    }

    // Перевірка ліміту розміру файлу (граничне значення: 50.0 МБ)
    if (FileSizeMB > 50.0)
    {
        throw new InvalidOperationException($"Файл '{FileName}' занадто великий ({FileSizeMB} МБ).
        Максимальний розмір — 50 МБ.");
    }

    // Перевірка допустимих розширень файлів (класи еквівалентності)
    if (Format != ".pdf" && Format != ".txt")
    {
        throw new NotSupportedException($"Формат {Format} не підтримується. Дозволені лише .pdf та
        .txt.");
    }
}

/// <summary>
/// Відповідає за процес обробки документа через зовнішню LLM API.
/// Керує статусом виконання та логікою повторних спроб (retry logic).
/// </summary>
public class ExtractionJob
{
    public Guid JobId { get; set; } = Guid.NewGuid();

    public string PromptText { get; set; }

    public string Status { get; set; } = "Pending";

    public Document TargetDocument { get; set; }

    /// <summary>

```

```

    /// Запускає процес екстракції даних із використанням механізму повторних спроб у разі збою мережі.

    /// </summary>

    /// <returns>Рядок із витягнутими даними у форматі JSON.</returns>

    /// <exception cref="TimeoutException">Викидається, якщо всі спроби підключення були
    невдалими.</exception>

    public async Task<string> ExecuteWithLLMAsync()
    {
        // Перед початком роботи обов'язково валідуємо документ
        TargetDocument.Validate();

        int maxRetries = 3; // Максимальна кількість спроб звернення до API

        int currentTry = 0;

        string extractedData = null;

        // Цикл повторних спроб для забезпечення надійності системи (NFR-R-02)
        while (currentTry < maxRetries)
        {
            currentTry++;

            try
            {
                Status = $"Processing (Attempt {currentTry}/{maxRetries})";

                Console.WriteLine(Status);

                // Симуляція мережевого запиту до зовнішнього API (наприклад, OpenAI)

                extractedData = await MockLLMCallAsync(PromptText);

                // Перевірка на порожню відповідь від API
                if (string.IsNullOrEmpty(extractedData))
                {
                    throw new Exception("LLM API повернула порожню відповідь.");
                }
            }
            catch { }
        }
    }

```

```

    }

    Status = "Completed";

    break; // Успішне виконання — достроковий вихід із циклу
}

catch (HttpRequestException ex)
{
    Console.WriteLine($"Помилка мережі на спробі {currentTry}: {ex.Message}");

    // Якщо це була остання спроба, перериваємо процес та викидаємо помилку

    if (currentTry == maxRetries)
    {
        Status = "Failed";

        throw new TimeoutException("Усі 3 спроби підключення до LLM API завершилися невдачею.",
ex);
    }

    // Затримка 2 секунди перед наступною спробою

    await Task.Delay(2000);
}

}

return extractedData;
}
}

/// <summary>

/// Відповідає за форматування та підготовку витягнутих даних до експорту.

/// </summary>

public class ExportFile
{

```

```

public Guid ExportId { get; set; } = Guid.NewGuid();

/// <summary>
/// Перетворює структуровані JSON-дані у формат CSV.
/// </summary>
/// <param name="jsonData">Дані у форматі JSON-масиву.</param>
/// <returns>Текстовий рядок у форматі CSV.</returns>
/// <exception cref="ArgumentNullException">Викидається, якщо вхідні дані відсутні.</exception>
/// <exception cref="FormatException">Викидається при неможливості розпарсити
JSON.</exception>

public string GenerateCSV(string jsonData)
{
    // Перевірка на порожні вхідні дані
    if (string.IsNullOrEmpty(jsonData))
    {
        throw new ArgumentNullException(nameof(jsonData), "Дані для експорту відсутні.");
    }

    try
    {
        // Десеріалізація вхідного JSON-рядка у список словників (ключ-значення)
        var data = JsonSerializer.Deserialize<List<Dictionary<string, string>>>(jsonData);

        // Перевірка на порожній масив (уникнення помилок у циклах)
        if (data == null || data.Count == 0) return string.Empty;

        var csvLines = new List<string>();

        // Витягування ключів з першого об'єкта для формування заголовків колонок CSV
        var headers = new List<string>(data[0].Keys);

```

```

csvLines.Add(string.Join(",", headers));

// Зовнішній цикл: обробка кожного рядка (об'єкта) з масиву
foreach (var row in data)
{
    var rowValues = new List<string>();

    // Вкладений цикл: обробка кожного значення за відповідним заголовком
    foreach (var header in headers)
    {
        // Безпечне отримання значення (якщо ключа немає, ставимо порожній рядок)
        string val = row.ContainsKey(header) ? row[header] : "";

        // Захист від пошкодження CSV: якщо текст містить кому, обгорнемо його в лапки
        if (val.Contains(",")) val = $"\"{val}\"";

        rowValues.Add(val);
    }

    // Формування рядка CSV через розділювач
    csvLines.Add(string.Join(",", rowValues));
}

// Об'єднання всіх рядків символом перенесення рядка
return string.Join("\n", csvLines);
}

catch (JsonException ex)
{
    // Перехоплення помилок синтаксису JSON та прокидання зрозумілої бізнес-помилки
    throw new FormatException("Помилка парсингу даних. Відповідь LLM має невалідну JSON-структуру.", ex);
}

```

```
}  
  
}
```

2. Таблиця проєктування тестів

Таблиця демонструє проєктування тестів за допомогою технік **розбиття на класи еквівалентності (EP)** та **аналізу граничних значень (BVA)**:

- Document.Validate():** Перевіряє бізнес-правила документа.
Головний акцент — тестування межі розміру файлу (BVA: 49.9 МБ, 50.0 МБ та відхилення на 50.1 МБ), а також допуск лише форматів .pdf та .txt.
- ExtractionJob.Execute...:** Тестує надійність циклу повторних спроб (Retry). Граничні значення (BVA): 1 спроба (ідеал), 3 спроби (допустимий максимум) та 4-та (відмова системи з викиданням TimeoutException).
- ExportFile.GenerateCSV():** Фокус на безпечному парсингу.
Перевіряється валідність JSON, межа масиву даних (0 елементів — повертає порожній рядок) та логіка форматування (якщо текст містить кому, він автоматично береться в лапки).

ID	Метод класу	Параметр / Умова, що тестується	Класи еквівалентності (EP)	Граничні значення (BVA)	Очікуваний результат
ТС-01.1	Document.Validate()	FileName (Ім'я файлу)	Валідний EP: Будь-який текст ("report.pdf"). Невалідний EP: null, "", " " (пробіли).	Не застосовується	Валідний: Продовження роботи. Невалідний: Викидається ArgumentException.

ID	Метод класу	Параметр / Умова, що тестується	Класи еквівалентності (ЕР)	Граничні значення (BVA)	Очікуваний результат
ТС-01.2	Document.Validate()	FileSizeMB (Розмір файлу)	Валідний ЕР: Від 0 до 50.0. Невалідний ЕР: Більше 50.0.	49.9 (Валідний) 50.0 (Валідна межа) 50.1 (Невалідна межа)	<= 50.0: Продовження роботи. 50.1: Викидається InvalidOperationException.
ТС-01.3	Document.Validate()	Format (Розширення файлу)	Валідний ЕР: ".pdf", ".txt". Невалідний ЕР: Будь-який інший (".docx", ".jpg").	Не застосовується	Валідний: Процес успішний. Невалідний: Викидається NotSupportedException.
ТС-02.1	ExtractionJob.Execute...	Кількість спроб підключення (HTTP)	Валідний ЕР: З'єднання встановлено за 1-3 спроби. Невалідний ЕР: 3 спроби завершилися тайм-аутом.	1 спроба (Мінімум) 3 спроби (Макс. межа) 4-та спроба (Відмова)	1-3: Метод повертає дані. Відмова (4): Викидається TimeoutException.

ID	Метод класу	Параметр / Умова, що тестується	Класи еквівалентності (EP)	Граничні значення (BVA)	Очікуваний результат
ТС-02.2	ExtractionJob.Execute...	Відповідь від LLM API	Валідний EP: Рядок із даними (JSON). Невалідний EP: null, порожній рядок.	Не застосовується	Валідний: Повертається рядок. Невалідний: Викидається базовий Exception.
ТС-03.1	ExportFile.GenerateCSV()	Вхідний параметр jsonStructuredData	Валідний EP: Рядок у форматі JSON. Невалідний EP: null, порожній рядок.	Не застосовується	Валідний: Перехід до десеріалізації. Невалідний: Викидається ArgumentException.
ТС-03.2	ExportFile.GenerateCSV()	Синтаксис JSON-рядка	Валідний EP: Валідна JSON-структура [{"K": "V"}]. Невалідний EP: Зламаний синтаксис (немає дужок тощо).	Не застосовується	Валідний: Дані конвертуються. Невалідний: Викидається FormatException.
ТС-03.3	ExportFile.GenerateCSV()	Розмір масиву даних	Валідний EP 1:	0 елементів	0: Метод повертає

ID	Метод класу	Параметр / Умова, що тестується	Класи еквівалентності (EP)	Граничні значення (BVA)	Очікуваний результат
		(JSON Array)	Порожній масив []. Валідний EP 2: Масив з елементами.	(Межа порожнечі) 1 елемент (Мін. робочий)	порожній рядок "". 1+: Формується валідний CSV-рядок.
ТС-03.4	ExportFile.GenerateCSV()	Обробка спецсимволів (коми у значеннях)	Валідний EP 1: Звичайний текст ("John"). Валідний EP 2: Текст із комою ("Kharkiv, UA").	Не застосовується	Текст із комою автоматично обгортається подвійними лапками: "Kharkiv, UA".

3-4. Вихідний код тестового набору з коментарями

Тестовий набір реалізовано з використанням сучасного constraint-орієнтованого синтаксису **NUnit 4** за шаблоном **AAA (Arrange, Act, Assert)**.

```
using System;
using System.Threading.Tasks;
using Xunit;
using XView.Core;

namespace XView.Tests
{
    public class DocumentTests
    {
        // TC-01: Техніка EP (Негативний) - Порожнє ім'я файлу
        [Theory]
        [InlineData(null)]
        [InlineData("")]
        [InlineData(" ")]
        public void Validate_EmptyFileName_ShouldThrowArgumentException(string badName)
        {
            // Arrange
```

```

        var doc = new Document { FileName = badName, FileSizeMB = 10.0, Format = ".pdf" };

        // Act
        Action act = () => doc.Validate();

        // Assert
        Assert.Throws<ArgumentException>(act);
    }

    // TC-02: Техніка BVA (Позитивний) - Максимально допустимий розмір
    [Theory]
    [InlineData(49.9)] // Трохи менше межі
    [InlineData(50.0)] // Точна межа
    public void Validate_ValidFileSize_ShouldNotThrowException(double fileSize)
    {
        // Arrange
        var doc = new Document { FileName = "report.pdf", FileSizeMB = fileSize, Format =
        ".pdf" };

        // Act
        var exception = Record.Exception(() => doc.Validate());

        // Assert
        Assert.Null(exception);
    }

    // TC-03: Техніка BVA (Негативний) - Перевищення ліміту розміру
    [Fact]
    public void Validate_FileSizeExceedsLimit_ShouldThrowInvalidOperationException()
    {
        // Arrange
        var doc = new Document { FileName = "big.pdf", FileSizeMB = 50.1, Format = ".pdf"
    };

        // Act
        Action act = () => doc.Validate();

        // Assert
        var ex = Assert.Throws<InvalidOperationException>(act);
        Assert.Contains("завеликий", ex.Message);
    }

    // TC-04: Техніка EP (Позитивний) - Допустимі формати файлів
    [Theory]
    [InlineData(".pdf")]
    [InlineData(".txt")]
    public void Validate_ValidFormat_ShouldNotThrowException(string validFormat)
    {
        // Arrange
        var doc = new Document { FileName = "doc" + validFormat, FileSizeMB = 5.0, Format
        = validFormat };

        // Act
        var exception = Record.Exception(() => doc.Validate());

        // Assert
        Assert.Null(exception);
    }

    // TC-05: Техніка EP (Негативний) - Недопустимі формати файлів
    [Theory]
    [InlineData(".docx")]
    [InlineData(".jpg")]
    public void Validate_InvalidFormat_ShouldThrowNotSupportedException(string
    invalidFormat)
    {
        // Arrange
        var doc = new Document { FileName = "image" + invalidFormat, FileSizeMB = 10.0,
        Format = invalidFormat };

        // Act
        Action act = () => doc.Validate();

        // Assert
        Assert.Throws<NotSupportedException>(act);
    }
}

```

```

    }

    public class ExtractionJobTests
    {
        // TC-06: Техніка EP (Негативний) - Валідація документа блокує виконання Job-a
        [Fact]
        public async Task ExecuteWithLLMAsync_DocumentIsInvalid_ThrowsExceptionBeforeExecution()
        {
            // Arrange
            var badDoc = new Document { FileName = "", FileSizeMB = 100.0, Format = ".exe" };
            var job = new ExtractionJob { TargetDocument = badDoc, PromptText = "Find names" };

            // Act
            Func<Task> act = async () => await job.ExecuteWithLLMAsync();

            // Assert
            // Має викинути ArgumentException через порожнє ім'я, ще до початку HTTP запитів
            await Assert.ThrowsAsync<ArgumentException>(act);
        }

        // TC-07: Техніка EP (Негативний) - Документ взагалі не передано (NullReference)
        [Fact]
        public async Task ExecuteWithLLMAsync_DocumentIsNull_ThrowsNullReferenceException()
        {
            // Arrange
            var job = new ExtractionJob { TargetDocument = null, PromptText = "Find names" };

            // Act
            Func<Task> act = async () => await job.ExecuteWithLLMAsync();

            // Assert
            await Assert.ThrowsAsync<NullReferenceException>(act);
        }
    }

    public class ExportFileTests
    {
        // TC-08: Техніка EP (Негативний) - Порожні вхідні дані для CSV
        [Theory]
        [InlineData(null)]
        [InlineData("")]
        public void GenerateCSV_EmptyData_ShouldThrowArgumentNullException(string emptyData)
        {
            // Arrange
            var export = new ExportFile();

            // Act
            Action act = () => export.GenerateCSV(emptyData);

            // Assert
            Assert.Throws<ArgumentNullException>(act);
        }

        // TC-09: Техніка EP (Негативний) - Невалідний синтаксис JSON
        [Fact]
        public void GenerateCSV_InvalidJsonSyntax_ShouldThrowFormatException()
        {
            // Arrange
            var export = new ExportFile();
            string badJson = "[{ Name: John"; // Зламаний синтаксис (без закриваючих дужок і
лапок)

            // Act
            Action act = () => export.GenerateCSV(badJson);

            // Assert
            Assert.Throws<FormatException>(act);
        }

        // TC-10: Техніка BVA (Позитивний) - Порожній масив JSON (0 елементів)
        [Fact]
        public void GenerateCSV_EmptyJsonArray_ShouldReturnEmptyString()
        {
            // Arrange
            var export = new ExportFile();

```

```

        string emptyJsonArray = "[]";

        // Act
        string result = export.GenerateCSV(emptyJsonArray);

        // Assert
        Assert.Equal(string.Empty, result);
    }

    // TC-11: Техніка ЕР (Позитивний) - Стандартний масив даних створє правильний CSV
    [Fact]
    public void GenerateCSV_ValidJsonArray_ShouldReturnCorrectCsvString()
    {
        // Arrange
        var export = new ExportFile();

        string jsonInput = "[{"Name\":\"John\", \"Role\":\"Admin\"}, {"Name\":\"Jane\", \"Role\":\"Dev\"}]";
        string expectedHeader = "Name,Role";
        string expectedRow1 = "John,Admin";
        string expectedRow2 = "Jane,Dev";

        // Act
        string result = export.GenerateCSV(jsonInput);

        // Assert
        Assert.Contains(expectedHeader, result);
        Assert.Contains(expectedRow1, result);
        Assert.Contains(expectedRow2, result);
    }

    // TC-12: Техніка ЕР (Позитивний) - Екранування ком всередині значень
    [Fact]
    public void GenerateCSV_ValueWithComma_ShouldWrapInQuotes()
    {
        // Arrange
        var export = new ExportFile();
        string jsonInput = "[{"City\":\"Kharkiv, UA\", \"Population\":\"1.4M\"}]";

        // Act
        string result = export.GenerateCSV(jsonInput);

        // Assert
        Assert.Contains("\"Kharkiv, UA\"", result); // Перевіряємо, що значення обгорнуте
    }
}

```

5-6. Скріншот покриття коду

Module	Line	Branch	Method
XView.Tests	57.83%	66.66%	85.71%
Total	57.83%	66.66%	85.71%
Average	57.83%	66.66%	85.71%

XView.Tests (тест) net10.0 успішно виконано (1,5 с)

Звіт згенеровано інструментами Coverlet. Загальний показник покриття рядків (Line Coverage): 85.71% Покриття розгалужень (Branch Coverage): 66.6%.

7. Висновки

Під час виконання лабораторної роботи було успішно закріплено практичні навички з розробки програмного коду на основі UML-моделей та проектування модульних тестів.

На базі діаграми класів системи X-View було реалізовано програмний модуль мовою C#, який містить нетривіальну бізнес-логіку обробки документів, взаємодії з API та експорту даних.

Для перевірки надійності модуля було застосовано техніки тест-дизайну: розбиття на класи еквівалентності (EP) та аналіз граничних значень (BVA). Це дозволило сформувати оптимальний набір із 11 тест-кейсів, які покривають як позитивні, так і негативні сценарії роботи системи.

За допомогою фреймворку xUnit було розроблено автоматизовані модульні тести з дотриманням патерну AAA (Arrange, Act, Assert). Аналіз покриття коду за допомогою інструменту Coverlet показав результат у 85%, що підтверджує високу якість тестування ключових алгоритмів (циклів, умовних конструкцій та блоків обробки винятків) і доводить стійкість розробленого програмного модуля до некоректних вхідних даних та системних збоїв.

